DESIGN, IMPLEMENT AND VALIDATE THE FARE LOGIC OF A CITY METRO SMART-CARD SYSTEM

C++ PROGRAMMING – ASSIGNMENT 1

Submitted by: Sahhanaa Shree Thiyagarajan
Register Number: 192511283A
Course Outcome: CO1 – Apply object-oriented programming principles and C++ constructs
Assignment Type: Individual
Total Marks: 50
Submission Date: 20 October 2026

GitHub Repository: https://github.com/sahhanaa44/DSA01-192511283A-Assignment1

ABSTRACT

The City Metro Smart-Card Fare System is designed to calculate metro fares accurately based on the travel zone, time of travel, and applicable concessions. The system uses a stored-value smart card, and a journey is permitted only when the remaining balance after fare deduction is at least ₹50.

This assignment explores the design, implementation, and validation of the fare calculation logic using C++. It covers data types, constants, operators, type conversions, control structures, functions, object-oriented programming, input validation, file handling, and testing. Different implementation approaches are compared to understand their advantages, limitations, time complexity, and space complexity.

The initial system supports three zones: Inner, Middle, and Outer. The fare values are ₹10, ₹20, and ₹30 respectively. A 20% surcharge is applied during peak hours, which are 8:00–10:00 AM and 5:00–8:00 PM. The final task extends the system with an Airport zone, weekend discounts, and monthly passes. Through these tasks, the assignment demonstrates how C++ programming techniques can be used to build a structured, accurate, and maintainable fare calculation system.

1. INTRODUCTION

Metro rail systems process a large number of passenger journeys every day. A smart-card system must calculate fares consistently, maintain accurate balances, and prevent invalid transactions. Errors in calculations or time-boundary conditions can result in incorrect charges or negative balances.

The Namma Metro Smart-Card Fare System considered in this assignment divides the metro network into three zones. Each zone has a fixed base fare, and peak-hour journeys incur an additional 20% surcharge. Passengers use stored-value smart cards, and the system must ensure that at least ₹50 remains in the card after the fare is deducted.

The assignment investigates common programming problems such as integer division, floating-point precision, signed and unsigned comparisons, operator precedence, duplicated fare values, and incorrect boundary checks. It then develops and tests alternative solutions using C++.

The work progresses from initial system analysis to data representation, fare calculation, input validation, testing, configuration, and system extension.

2. OBJECTIVES

The main objectives of this assignment are:

1. To analyse the requirements of a metro smart-card fare system.
2. To identify the main objects, data, and operations involved in the system.
3. To select suitable C++ data types for money, time, zones, and balances.
4. To use named constants instead of repeatedly writing fare values.
5. To understand implicit conversions, explicit conversions, and operator precedence.
6. To implement peak-hour fare calculation using conditional statements and functions.
7. To compare procedural programming and object-oriented programming.
8. To validate user input and format monetary output correctly.
9. To test normal cases, boundary conditions, and invalid inputs.
10. To calculate journey totals accurately and analyse possible overflow.
11. To store fare configuration in an external file.
12. To extend the fare system with an Airport zone, weekend discounts, and monthly passes.

3. PROBLEM STATEMENT

The metro smart-card system must calculate the fare based on the farthest zone travelled. The initial fare structure is:

Zone	Base Fare
Inner	₹10
Middle	₹20
Outer	₹30

A 20% surcharge applies during peak hours:

- Morning peak: 8:00 AM to before 10:00 AM.
- Evening peak: 5:00 PM to before 8:00 PM.

The start of each interval is included, while the end is excluded. Therefore, 8:00 AM and 5:00 PM are peak times, whereas 10:00 AM and 8:00 PM are off-peak times.

The system must also consider student and senior-citizen concessions, validate the selected zone and time, prevent invalid fare deductions, and ensure that at least ₹50 remains on the smart card after an ordinary journey.

In the final extension, the system includes an Airport zone, weekend discounts, and monthly passes that cover two selected zones.

4. SYSTEM ANALYSIS

4.1 Existing System

An initial fare calculation program can become difficult to maintain when fare values are repeated in multiple places. If a fare changes, every occurrence must be updated manually. Incorrect type conversions can also produce inaccurate results, while poorly defined peak-hour conditions can classify boundary times incorrectly.

Using floating-point values for money may introduce precision problems. In addition, an incorrect balance deduction can allow a card to fall below the required minimum balance.

4.2 Proposed System

The proposed system uses named constants, integer-based monetary values, well-defined conditions, and reusable functions. It validates inputs before performing calculations and uses a structured approach to testing.

Object-oriented programming is used to demonstrate how a smart-card class can protect the balance and control deductions. Later tasks explore configuration files and additional fare rules.

4.3 Main System Components

The important components are:

- **Zone:** Represents the travel zone selected by the passenger.
- **Fare:** Stores and calculates the cost of a journey.
- **Peak-hour checker:** Determines whether a journey occurs during peak hours.
- **Smart card:** Maintains the passenger's balance.
- **Fare calculator:** Applies the base fare, surcharge, and concession.
- **Input handler:** Reads and validates user input.
- **Configuration file:** Stores fare values and percentages.
- **Monthly pass:** Represents access to the zones covered by a pass.
- **Test cases:** Verify that calculations and boundary conditions are correct.

5. DESIGN AND IMPLEMENTATION

TASK 1 – SYSTEM ANALYSIS AND OBJECT IDENTIFICATION

Aim

To analyse the metro smart-card fare system, identify its main objects, and determine the required data and operations.

Analysis

The system contains several related entities. The passenger selects a destination zone and provides the required journey details. The fare calculator determines the applicable amount, while the smart card maintains the available balance.

The main data items include the zone, base fare, journey hour, surcharge percentage, concession percentage, and card balance. The main operations include calculating the fare, checking peak hours, validating the journey, deducting the fare, and recharging the card.

The system can be represented using a class diagram showing the relationship between the smart card, fare calculator, zone information, and journey details. The design separates the fare calculation from the balance-management logic, making the program easier to understand and modify.

Result

The system requirements and major components were identified. The analysis provides the foundation for selecting appropriate data types and implementing the fare calculation logic.

Evidence to insert: UML class diagram and Task 1 analysis screenshot.

TASK 2 – CONSTANTS, ENUMERATIONS, AND MACROS

Aim

To represent fixed fare values and system rules using named constants and compare enumerations with macros.

Analysis

Repeated numeric values make programs difficult to maintain. Named constants provide a central location for storing values such as base fares, surcharge percentages, and minimum balance requirements.

The initial fare values are stored in paise:

- Inner zone: 1000 paise
- Middle zone: 2000 paise
- Outer zone: 3000 paise
- Peak surcharge: 20%
- Minimum remaining balance: 5000 paise

The peak-hour boundaries are represented using named integer constants. This avoids repeating the same hour values throughout the program.

An enumeration such as `enum class Zone` provides a clear representation of the available zones. It also improves type safety because values from the enumeration cannot be freely mixed with unrelated integers without an explicit conversion.

Macros created using `#define` perform textual substitution. They do not provide the same type checking as a scoped enumeration or a typed constant. Therefore, named constants and `enum class` are generally easier to maintain and safer for representing the fare system.

Result

Named constants and enumerations provide a structured way to represent fare rules and zones. The comparison demonstrates why typed constants and scoped enumerations are preferable to unrestricted macros for this application.

Evidence to insert: Output screenshots for `main.cpp` and `macro_version.cpp`.

TASK 3 – DATA TYPES AND NUMERIC PRECISION

Aim

To evaluate C++ data types and determine suitable types for monetary calculations and journey totals.

Analysis

Different data types have different ranges and levels of precision. The `int` type is suitable for values such as hours and zone indexes, while `bool` is useful for conditions such as whether a journey occurs during peak hours.

The `double` type can represent fractional values, but decimal fractions such as 0.1 cannot always be represented exactly in binary floating-point format. Repeated floating-point calculations may therefore produce small precision differences.

For monetary calculations, the system uses `long long` to store integer values in paise. Since ₹1 equals 100 paise, ₹10 is represented as 1000 paise. This avoids fractional currency values during the main fare calculation.

The task also examines the maximum values of integer types and the size of accumulated journey totals. A network processing millions of journeys may generate totals that exceed the range of a normal 32-bit integer. Using a wider integer type helps accommodate larger totals.

Result

Integer-based monetary representation using `long long` paise is suitable for exact fare calculations. The task also demonstrates the importance of checking data-type limits when accumulating large numbers of transactions.

Evidence to insert: Task 3 program output showing type sizes, numeric limits, and precision comparison.

TASK 4 – TYPE CONVERSIONS AND DEBUGGING

Aim

To identify errors caused by implicit type conversions and correct them using explicit conversions.

Analysis

C++ allows values to be converted automatically between different data types. However, implicit conversions can sometimes produce unexpected results.

For example, when two integers are divided, integer division is performed before the result is stored in a `double`. Therefore, the expression `7 / 2` produces `3`, not `3.5`. Converting one operand to `double` before division produces the expected fractional result.

Characters are internally represented by numeric character codes. Adding an integer to a character changes its character code, which can produce another character when printed as a character or a number when promoted to an integer.

A Boolean variable assigned a non-zero value becomes `true`. Its default numeric output is `1`, while `std::boolalpha` prints the word `true`.

Signed and unsigned integer comparisons can also be misleading. A negative signed integer may be converted to a large unsigned value during comparison, causing an expression to behave differently from what is expected.

Explicit conversions using `static_cast` make the programmer's intention clearer. However, the conversion must be chosen carefully because converting a value does not automatically make an incorrect calculation safe.

Result

The task demonstrates how implicit conversions can cause unexpected outputs and how explicit conversions can make arithmetic and comparisons clearer and more predictable.

Evidence to insert: Predicted output, original program output, and corrected program output.

TASK 5 – OPERATOR PRECEDENCE AND PEAK-HOUR CONDITIONS

Aim

To analyse operator precedence in fare calculations and validate peak-hour boundaries.

Analysis

Operator precedence determines the order in which parts of an expression are evaluated. Parentheses can be used to make calculations clearer and reduce the risk of mistakes.

For a base fare of 1000 paise and a surcharge of 20%, the correct calculation is:

Surcharge = 1000 × 20 / 100 = 200 paise

Final fare = 1000 + 200 = 1200 paise

An incorrectly grouped expression may calculate a different result. The task compares the correct and incorrect expressions to demonstrate why the order of operations matters.

Peak-hour logic is implemented using logical AND and OR operators. The morning interval is defined as `hour >= 8 && hour < 10`, while the evening interval is `hour >= 17 && hour < 20`.

These conditions include the starting hour and exclude the ending hour. Testing the hours 7, 8, 9, 10, 16, 17, 19, and 20 verifies the boundaries and ordinary off-peak cases.

Result

The task demonstrates the importance of operator precedence and clearly defined boundary conditions. Correct grouping and consistent peak-hour rules prevent incorrect surcharge calculations.

Evidence to insert: Output screenshots for `precedence.cpp` and `peak.cpp`.

TASK 6 – FARE CALCULATION USING CONTROL STRUCTURES

Aim

To implement fare calculation using a switch statement and compare it with an array-based lookup table.

Analysis

A switch statement can select the base fare based on the chosen zone. Each case represents one zone and assigns its corresponding fare. Conditional statements then apply the peak surcharge and concession when required.

An alternative implementation stores the base fares in an array. The zone index is used to retrieve the appropriate fare. This separates the fare data from the calculation logic.

Both approaches provide constant-time lookup for a fixed number of zones. However, the array-based approach can make fare data easier to manage when more zones are introduced, provided the index is validated before accessing the array. A switch statement can be clear for a small number of distinct cases, but additional zones require additional branches.

The implementations use integer-based fare calculations. The peak surcharge is applied using the configured percentage, and the concession is applied according to the assumption stated in this report.

Result

Both control-structure approaches demonstrate how the fare can be selected and adjusted. The comparison highlights the trade-off between explicit branching and data-driven lookup.

Evidence to insert: Pseudocode and output screenshots for `fare_switch.cpp` and `fare_table.cpp`.

TASK 7 – PROCEDURAL PROGRAMMING AND OBJECT-ORIENTED PROGRAMMING

Aim

To implement the fare logic using procedural and object-oriented approaches and compare their maintainability.

Analysis

In procedural programming, the fare calculation is implemented through functions and separate variables. The calculation receives the zone, peak-hour status, and concession status, then returns the fare.

In object-oriented programming, related data and operations can be grouped into a class. A `SmartCard` class can store the balance as a private member and provide methods such as `canTravel()` and `deduct()`.

Keeping the balance private prevents unrelated parts of the program from modifying it directly. The deduction method can check whether the fare is valid and whether the remaining balance satisfies the minimum balance rule before updating the balance.

If the balance were public, any part of the program could change it without following the validation rules. This could allow invalid or negative balances. Private data and controlled methods help enforce the system's rules.

Both approaches can calculate a fare in constant time. The main difference is not performance but how the program organises data, controls access, and supports future changes.

Result

The procedural version provides a simple and direct calculation, while the object-oriented version offers better encapsulation for smart-card balance management. The object-oriented approach is more suitable when the system grows to include additional card operations and transaction rules.

Evidence to insert: Output screenshots for `procedural.cpp` and `oop.cpp`.

TASK 8 – INPUT VALIDATION AND FORMATTED OUTPUT

Aim

To read passenger details safely, handle invalid input, and display monetary values clearly.

Analysis

Input validation is important because users may enter letters where numbers are expected or provide values outside the valid range.

Using `cin` allows numeric input to be read directly. If the input fails, the program can check the stream state, clear the error using `cin.clear()`, and discard the invalid characters using `cin.ignore()`. The program can then ask the user to enter the value again.

An alternative is to read an entire line using `getline()` and convert it using `stoi()` inside a `try/catch` block. This method can handle conversion errors, but the program must also check whether the entire input was consumed. For example, an input such as `12abc` may initially convert to the integer 12 unless the remaining characters are checked.

The `<iomanip>` library provides formatting tools such as `setw`, `setfill`, `fixed`, and `setprecision`. These tools help present monetary values in a consistent format.

The two approaches are compared in terms of input handling and buffer usage. Processing a line of length L takes O(L) time in both approaches, while line-based input requires storage for the line.

Result

The task demonstrates input validation, recovery from invalid values, and formatted output. Safe input handling reduces the possibility of invalid zone or time values entering the fare calculation.

Evidence to insert: Output screenshots for `io_cin.cpp` and `io_getline.cpp`.

TASK 9 – TESTING AND VALIDATION OF FARE LOGIC

Aim

To validate the fare calculation using test cases covering normal journeys, peak hours, concessions, and invalid values.

Analysis

Testing verifies whether the program produces the expected fare for different inputs. A test table makes it easier to compare expected results with actual results and identify errors.

The following representative cases are used:

Test case	Input condition	Expected fare
1	Inner, off-peak, no concession	₹10
2	Middle, off-peak, no concession	₹20
3	Outer, peak, no concession	₹36
4	Middle, off-peak, 50% concession	₹10
5	Invalid zone	Invalid result

Additional boundary tests check 8:00, 10:00, 17:00, and 20:00. These confirm that the start of a peak interval is included and its end is excluded.

Balance validation is also important. If the balance is ₹60 and the fare is ₹10, the remaining balance is exactly ₹50, so the journey satisfies the minimum balance rule. If the balance is ₹59.99 and the fare is ₹10, the remaining balance is ₹49.99, so the journey must be rejected.

A small set of five journeys can be added manually to verify the total. For example, fares of ₹10, ₹24, ₹30, ₹36, and ₹10 total ₹110. The test program compares expected and actual values and reports whether each case passes.

Result

Testing helps identify errors in fare calculation, time boundaries, invalid inputs, and minimum balance checks. A table-driven test approach makes it easier to add new test cases and repeat the validation after changes.

Evidence to insert: Test output showing PASS/FAIL results and the test-case table.

TASK 10 – MONEY REPRESENTATION, TOTALS, AND COMPILER WARNINGS

Aim

To calculate journey totals accurately, validate monetary inputs, and use compiler warnings to identify potential programming issues.

Analysis

The total fare for multiple journeys is calculated by adding each fare to an accumulated total. Because the number of journeys may grow significantly, the accumulator must have sufficient range.

The system stores monetary values in paise using `long long`. This avoids the floating-point representation issues associated with repeatedly adding decimal currency values. The final total can be converted into rupees and paise for display.

The program validates the number of fares and rejects negative fare inputs. These checks prevent invalid values from being included in the total.

Compiler options such as `-Wall`, `-Wextra`, `-Wconversion`, and `-pedantic` help identify suspicious conversions, unused values, and other potential issues. A clean compiler output indicates that no warnings were reported for that compilation, but it does not prove that the program is completely free of logical errors. Runtime testing is still necessary.

The program processes each fare once, so its time complexity is O(n), where n is the number of fares. It uses a constant amount of additional working memory, giving O(1) auxiliary space.

Result

The task demonstrates exact monetary accumulation, input validation, and the use of compiler warnings. These practices improve reliability when the number of journeys becomes large.

Evidence to insert: Program output and compiler-warning output or the generated `warnings.txt` file.

TASK 11 – CONFIGURATION FILES AND DESIGN RECOMMENDATIONS

Aim

To store fare settings in an external configuration file and evaluate the benefits of configurable system values.

Analysis

Hard-coding every fare value in a program makes updates more difficult. An external configuration file allows the fare values and percentages to be changed without editing the main calculation logic.

The configuration file used in this task stores the base fares for the three zones, the peak surcharge percentage, and the concession percentage. The program reads these values and uses them during fare calculation.

For example, the configuration file contains 1000, 2000, and 3000 paise for the three base fares, followed by the 20% peak surcharge and the assumed 50% concession.

Separating configuration data from the program makes the system easier to maintain. If the fare structure changes, the configuration can be updated instead of searching through several source files.

However, configuration input must be validated. A practical system should check whether the file opened successfully, whether every expected value was read, and whether the values fall within acceptable ranges. Invalid or missing configuration data should be reported rather than silently used.

Result

The task demonstrates that configuration files improve maintainability by separating fare settings from calculation logic. Input validation is necessary to ensure that the configured values are complete and valid.

Evidence to insert: Contents of `fares.cfg`, program output, and the recommendation analysis.

TASK 12 – EXTENDING THE SYSTEM WITH NEW FARE RULES

Aim

To evaluate and extend the fare system with an Airport zone, weekend discounts, and monthly passes.

Analysis

The final task expands the original three-zone system by adding an Airport zone with a base fare of ₹60. The system therefore supports four zones:

Zone	Base fare
Inner	₹10
Middle	₹20
Outer	₹30
Airport	₹60

The peak surcharge remains 20%. For off-peak journeys on weekends, a 10% discount is applied. In the implementation, peak-hour surcharge takes priority over the weekend discount, so the weekend discount is not applied to peak journeys. This is an implementation assumption because the original scenario does not define how these two rules interact.

A monthly pass can cover two selected zones. If a journey is within a zone covered by the pass, the fare can be waived according to the implemented pass rule. The `MonthlyPass` class stores the two zones and checks whether the selected journey zone is covered. The `SmartCard` class can use this information before deducting a normal fare.

The design uses a table of base fares and separate conditions for peak and weekend adjustments. This approach reduces the need to duplicate fare logic for every zone. The pass logic is kept separate from the normal fare calculation.

The test cases check the Airport fare, weekend fare, peak surcharge, the interaction between peak and weekend conditions, and invalid zones. The test results are used to evaluate whether the extension behaves as expected.

The implementation represents a working model of the extended rules. A complete production system would also need to validate monthly-pass eligibility and validity dates, handle card transactions safely, and ensure that the pass covers only the zones selected by the passenger.

Result

The fare system was extended to include an Airport zone, weekend discounts, and monthly-pass handling. The use of a fare table and separate classes makes the extended system easier to maintain than duplicating the original fare calculation for each new rule.

Evidence to insert: Output screenshots for `metro_v2.cpp` and `test_v2.cpp`, along with the impact analysis.

6. CONSOLIDATED SYSTEM RULES AND ASSUMPTIONS

The implementation uses the following rules consistently wherever applicable:

1. The initial system contains Inner, Middle, and Outer zones with base fares of ₹10, ₹20, and ₹30.
2. Money is stored as integer paise using `long long`.
3. A 20% surcharge applies during 8:00–10:00 AM and 5:00–8:00 PM, with the ending times excluded.
4. The minimum remaining balance for an ordinary journey is ₹50.
5. A 50% concession is used for students and senior citizens as an implementation assumption, because the original scenario does not specify the exact concession rate.
6. The extended system adds an Airport zone with a base fare of ₹60.
7. The extended system applies a 10% weekend discount to off-peak journeys. Peak surcharge takes priority over the weekend discount in this implementation.
8. The monthly-pass implementation treats a journey in a covered zone as free. The validity and payment details of a real monthly pass are outside the scope of this basic implementation.

These assumptions should be clearly distinguished from the rules explicitly stated in the assignment scenario.

7. TESTING STRATEGY

The system is tested using normal, boundary, invalid, and extended-rule cases.

7.1 Normal Cases

Normal cases verify that the base fare for each valid zone is calculated correctly during off-peak hours.

7.2 Peak-Hour Boundary Cases

The hours 8, 10, 17, and 20 are tested to confirm the exact peak-hour boundaries. The hours 8 and 17 are peak, while 10 and 20 are off-peak.

7.3 Concession Cases

Concession tests check whether the configured concession percentage is applied correctly to the fare.

7.4 Invalid Input Cases

Invalid zone values, out-of-range hours, non-numeric input, negative fares, and invalid configuration values should be rejected or handled safely.

7.5 Minimum Balance Cases

The balance rule is checked using a case where the remaining balance is exactly ₹50 and another case where it falls below ₹50.

7.6 Extended System Cases

The extended system is tested for the Airport fare, weekend discount, peak surcharge, peak-weekend interaction, and monthly-pass coverage.

Testing is important because a program can compile successfully and still contain logical errors. Both expected and actual outputs must be compared before the system is considered validated.

8. TIME AND SPACE COMPLEXITY ANALYSIS

Time complexity describes how the amount of work increases with the input size. Space complexity describes the additional memory required.

Component	Time complexity	Auxiliary space	Explanation

Zone lookup using switch	O(1)	O(1)	A fixed number of branches
Zone lookup using an array	O(1)	O(1)	Direct indexed lookup
Peak-hour check	O(1)	O(1)	A fixed number of comparisons
Single fare calculation	O(1)	O(1)	A fixed number of arithmetic operations
Input line processing	O(L)	O(L)	L is the length of the input line
Total of n fares	O(n)	O(1)	Each fare is processed once
Test table with t cases	O(t)	O(t) for the stored test table	Each test is checked in a loop
Monthly-pass zone check	O(1)	O(1)	A fixed number of zone comparisons

The switch and array approaches both provide constant-time fare lookup for the fixed zone structure. The main difference is maintainability and how new zones are represented, rather than a significant difference in asymptotic complexity.

The procedural and object-oriented versions also have the same basic time and space complexity for a single fare calculation. Object-oriented programming mainly improves encapsulation and organisation; it does not automatically make the calculation faster.

9. OVERALL OBSERVATIONS

The tasks demonstrate that accurate fare calculation depends on more than writing arithmetic expressions. Data representation, validation, clear boundary rules, and maintainable design are equally important.

Using integer paise avoids common floating-point issues in monetary calculations. Named constants prevent fare values from being repeated unnecessarily, while scoped enumerations provide a clearer representation of zones.

Explicit type conversions help prevent unexpected results. Parentheses and clearly defined conditions improve the reliability of fare expressions. Input validation prevents invalid values from reaching the calculation logic, and test tables make it easier to verify the system after changes.

Procedural functions are useful for straightforward calculations, while classes are useful when data and operations must be kept together. Configuration files help separate fare data from source code, and the extended fare model demonstrates how a table-based design can support additional zones and rules.

The final extension also highlights the need to document assumptions. Concession rates, the interaction between peak surcharges and weekend discounts, and monthly-pass validity must be specified clearly before a real system can be deployed.

10. LIMITATIONS AND FUTURE ENHANCEMENTS

The programs developed for this assignment demonstrate the core fare calculation concepts, but they are simplified educational implementations.

Possible future improvements include:

1. Adding a complete passenger registration and smart-card management module.
2. Storing card balances and journey histories in a database.
3. Implementing secure recharge and fare-deduction transactions.
4. Adding stronger validation for missing or malformed configuration files.
5. Defining the concession eligibility rules and the exact discount policy.
6. Validating monthly-pass dates, passenger eligibility, and the two covered zones.
7. Recording transaction receipts and providing journey reports.
8. Adding more zones through a configuration-driven design.
9. Expanding automated tests to cover every boundary, invalid input, and balance condition.
10. Measuring performance using larger input datasets to support complexity analysis with practical observations.

These improvements would move the program closer to a complete metro smart-card application while preserving the core design principles explored in the assignment.

11. CONCLUSION

This assignment presented the analysis, implementation, and validation of the fare logic for a City Metro Smart-Card System using C++. The twelve tasks covered system analysis, constants, enumerations, data types, type conversions, operator precedence, control structures, procedural programming, object-oriented programming, input validation, testing, monetary totals, configuration files, and system extensions.

The work demonstrated the importance of storing money as integer paise, defining peak-hour boundaries consistently, validating input, and checking that a card retains the required minimum balance. It also showed that named constants improve maintainability, test tables support systematic validation, and encapsulation helps protect the smart-card balance.

The final extension introduced an Airport zone, weekend discounts, and monthly passes, demonstrating how a structured design can accommodate additional business rules. Overall, the assignment strengthened the understanding of C++ fundamentals and showed how programming concepts can be applied to a practical fare calculation problem.

12. REFERENCES

1. C++ Programming – Assignment 1: *Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System*, SIMATS Online Lab.
2. C++ standard library documentation for input/output streams, numeric limits, formatting, and file handling.
3. Compiler documentation for the GNU C++ compiler (g++) and compiler warning options.

APPENDIX A – PROGRAM EXECUTION EVIDENCE

- Task 1: System analysis and UML class diagram.

```
PS C:\DSA01-192511283A-Assignment1> g++ -std=c++17 -Wall -Wextra Task02/main.cpp -o Task02/main
PS C:\DSA01-192511283A-Assignment1> .\Task02\main.exe
Metro Smart-Card System
Zone: Inner
Base fare: 1000 paise
Peak surcharge: 20%
Minimum balance: 5000 paise
```

- Task 2: Constants and macro comparison.

```
PS C:\DSA01-192511283A-Assignment1> g++ -std=c++17 -Wall -Wextra Task03/main.cpp -o Task03/main
PS C:\DSA01-192511283A-Assignment1> .\Task03\main.exe
Size of int: 4 bytes
Size of long long: 8 bytes
Maximum int: 2147483647
Maximum long long: 9223372036854775807
Sum of 0.1 ten times: 1
Fare in paise: 3000
```

- Task 3: Data types, numeric limits, and precision.

```
PS C:\DSA01-192511283A-Assignment1> g++ -std=c++17 -Wall -Wextra Task04/conversions.cpp -o Task04/conversions
Task04/conversions.cpp: In function 'int main()':
Task04/conversions.cpp:19:21: warning: comparison between signed and unsigned integer expressions [-Wsign-compare]
     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
                   ~~^~~
PS C:\DSA01-192511283A-Assignment1> .\Task04\conversions.exe
3 3.5
C 68
1 true
s >= u
PS C:\DSA01-192511283A-Assignment1> g++ -std=c++17 -Wall -Wextra Task04/conversions_fixed.cpp -o Task04/conversions_fi
xed
PS C:\DSA01-192511283A-Assignment1> .\Task04\conversions_fixed.exe
3 3.5
C 68
1 true
s < u
```

- Task 4: Original and corrected type-conversion programs.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task05/peak.cpp -o Task05/peak.exe
PS C:\DSA01-192511283A-Assignment1> .\Task05\peak.exe
7:00 - Off-peak
8:00 - Peak
9:00 - Peak
10:00 - Off-peak
16:00 - Off-peak
17:00 - Peak
19:00 - Peak
20:00 - Off-peak
```

- Task 5: Operator precedence and peak-hour boundary tests.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task06/fare_table.cpp -o Task06/fare_table.exe
PS C:\DSA01-192511283A-Assignment1> .\Task06\fare_table.exe
1. Inner  2. Middle  3. Outer
Enter zone: 2
Peak hour? (1-Yes, 0-No): 1
Concession? (1-Yes, 0-No): 0
Final fare: 2400 paise
PS C:\DSA01-192511283A-Assignment1> g++ Task06/fare_switch.cpp -o Task06/fare_switch.exe
PS C:\DSA01-192511283A-Assignment1> .\Task06\fare_switch.exe
1. Inner  2. Middle  3. Outer
Enter zone: 1
Peak hour? (1-Yes, 0-No): 1
Concession? (1-Yes, 0-No): 0
Final fare: 1200 paise
PS C:\DSA01-192511283A-Assignment1>
```

- Task 6: Switch-based and array-based fare calculations.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task07/procedural.cpp -o Task07/procedural.exe
PS C:\DSA01-192511283A-Assignment1> .\Task07\procedural.exe
Enter zone (1-Inner, 2-Middle, 3-Outer): 3
Peak hour? (1-Yes, 0-No): 0
Concession? (1-Yes, 0-No): 1
Final fare: 1500 paise
PS C:\DSA01-192511283A-Assignment1>
PS C:\DSA01-192511283A-Assignment1> g++ Task07/oop.cpp -o Task07/oop.exe
PS C:\DSA01-192511283A-Assignment1> .\Task07\oop.exe
Enter zone (1-Inner, 2-Middle, 3-Outer): 3
Peak hour? (1-Yes, 0-No): 1
Concession? (1-Yes, 0-No): 0
Final fare: 3600 paise
```

- Task 7: Procedural and object-oriented implementations.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task08/io_cin.cpp -o Task08/io_cin.exe
PS C:\DSA01-192511283A-Assignment1> .\Task08\io_cin.exe
Enter passenger name (one word): sahana
Enter zone (1-Inner, 2-Middle, 3-Outer): 2
Passenger: sahana
Zone: 2
PS C:\DSA01-192511283A-Assignment1>
PS C:\DSA01-192511283A-Assignment1> g++ Task08/io_getline.cpp -o Task08/io_getline.exe
PS C:\DSA01-192511283A-Assignment1> .\Task08\io_getline.exe
Enter passenger full name: sahana t
Enter zone (1-Inner, 2-Middle, 3-Outer): 2
Passenger: sahana t
Zone: 2
```

- Task 8: Input validation and formatted output.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task09/test_fare.cpp -o Task09/test_fare.exe
PS C:\DSA01-192511283A-Assignment1> .\Task09\test_fare.exe
Inner off-peak: PASS (Expected 1000, Got 1000)
Middle off-peak: PASS (Expected 2000, Got 2000)
Outer peak: PASS (Expected 3600, Got 3600)
Middle concession: PASS (Expected 1000, Got 1000)
Invalid zone: PASS (Expected -1, Got -1)
```

- Task 9: Fare test cases and PASS/FAIL results.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task10/totals.cpp -o Task10/totals.exe
PS C:\DSA01-192511283A-Assignment1> .\Task10\totals.exe
Enter number of passengers: 4
Enter fare for passenger 1 in paise: 5
Enter fare for passenger 2 in paise: 6
Enter fare for passenger 3 in paise: 7
Enter fare for passenger 4 in paise: 8
Total collected: 26 paise
Total collected: 0 rupees and 26 paise
```

- Task 10: Fare totals and compiler-warning output.

```
PS C:\DSA01-192511283A-Assignment1> g++ Task11/config_fare.cpp -o Task11/config_fare.exe
PS C:\DSA01-192511283A-Assignment1> .\Task11\config_fare.exe
Enter zone (1-3): 3
Peak hour? (1-Yes, 0-No): 0
Concession? (1-Yes, 0-No): 0
Final fare: 3000 paise
```

- Task 11: Configuration file and program output.

```
PS C:\DSA01-192511283A-Assignment1> .\Task12\test_v2.exe
Inner normal: PASS (Expected 1000, Got 1000)
Airport normal: PASS (Expected 6000, Got 6000)
Airport peak: PASS (Expected 7200, Got 7200)
Middle weekend: PASS (Expected 1800, Got 1800)
Peak overrides weekend: PASS (Expected 2400, Got 2400)
Invalid zone: PASS (Expected -1, Got -1)
```

- Task 12: Extended fare calculation and test results.

APPENDIX B – GITHUB SUBMISSION

Repository name: `DSA01-192511283A-Assignment1`

Repository link: https://github.com/sahhanaa44/DSA01-192511283A-Assignment1

The repository should contain the source code, analysis files, input/configuration files, output screenshots, and README for the individual tasks. The README should identify the compiler and version, provide the compilation and execution commands, and summarise the result of each task.

The commit history should show progressive development, with at least one commit for each task, as required by the assignment.